

complexity is $O(N)$. The interesting question for us is whether it is possible to find the maximum by doing fewer than $(N-1)$ comparisons. We can show the lower bound by using an adversary argument.

The idea behind an adversary argument is that we start off with an unspecified input and whenever the algorithm probes the input, the adversary gives an answer such that the algorithm is forced to work hard. The adversary must be consistent in its answers in the sense that there must exist at least one set of valid inputs on which the algorithm will exactly see the adversary answers matching with that input. The adversary's goal is to keep the algorithm uncertain of the answer for as long as possible. If the algorithm declares that it has found the answer without proceeding through the established lower bound number of probes, the adversary can always come up with a sequence of inputs that will invalidate the algorithm's answer.

For instance, consider a trivial example of a sequence of four numbers, namely $\{1, 2, 3, X\}$ and the algorithm needs to find the maximum. Assume that the algorithm performed two comparisons only, namely, between 1 and 2, and 2 and 3. It skipped the comparison of the current maximum 3 with the last element X in the array and declared the answer as 3. Since the algorithm has not probed $A[3]$, the adversary can fill it up with

any value for X , thereby defeating the algorithm. No matter how the algorithm tries, if it does not probe one of the elements of the array, the adversary can fill up that element with a value that will defeat the answer provided by the algorithm. Thus an adversary argument shows that any algorithm for finding the maximum in an array of elements has a lower bound of $O(N)$ comparisons for arriving at the correct answer.

For this month's takeaway problem, consider the well-known problem of deciding whether a given graph with N nodes is connected. The only question the algorithm can ask the adversary is of the form, "Does an edge exist between vertex u and vertex v ?" What is the best lower bound you can establish for this algorithm using an adversary argument?

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me, along with your feedback, to sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming! 

About the author:

Sandya Mannarswamy. The author is a specialist in compiler optimisation and works at Hewlett-Packard India. She has a number of publications and patents to her credit, and her areas of interest include virtualisation technologies and software development tools.



Calling...

Share your knowledge

Inviting FOSS experts to write articles on their area of interest

LFX covers a myriad of topics—network management, software development, embedded systems, community issues, and even hands-on guide for newbies. If you've got an interesting topic, let us know. Thanks to the launch of www.openTis.com (aka linuxforu.com) we are now trying to extend our content portfolio related to Linux & Open Source.

To know more on how to become an LFX author, contact us at

lfyedit@efyindia.com

Few topics that top our list:

- ◆ Tips 'n' Tricks for software developets or IT implementers
- ◆ Cool tweaks for FOSS enthusiasts
- ◆ FOSS on mobile
- ◆ Virtualisation (Implementation)
- ◆ OpenJDK or Java on Linux
- ◆ OpenSolaris (software development)
- ◆ How can I do 'that' on Linux
- ◆ Reviews of latest open source projects & tools